In-Class Assignment: Linear Regression

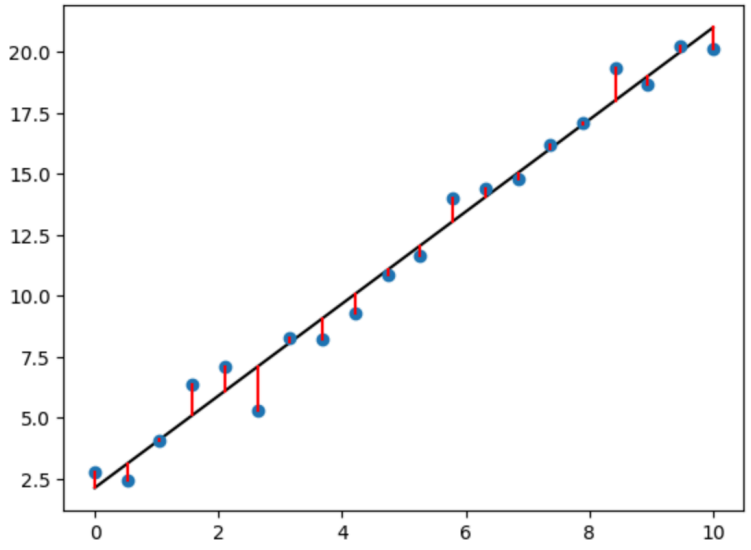
Please answer the following questions.

Question 1: Simple Linear Regression Parameters
In the first linear regression example in the notebook (the code block with model=LinearRegression(fit_intercept=True)), what do model.coef_ and model.intercept_ represent? Also, explain which parts of the graph they correspond to.

Question 2: Sum of Squared Errors
The first linear regression plot shows red lines connecting the original data points to the regression line. What do these red lines represent? And what does the LinearRegression model try to minimize in relation to these red lines?

```
model=LinearRegression(fit_intercept=True)
model.fit(x[:,np.newaxis], y)
xfit=np.linspace(0,10,100)
yfit=model.predict(xfit[:, np.newaxis])
plt.plot(xfit,yfit, color="black")
plt.plot(x,y, 'o')
# The following will draw as many line segments as there are columns in matrices x and y
plt.plot(np.vstack([x,x]), np.vstack([y, model.predict(x[:, np.newaxis])]), color="red");
```

```
print("Parameters:", model.coef_, model.intercept_)
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
```



Question 3: Multiple Features and Intercept
In the 'Multiple features' section, the code model2=LinearRegression(fit_intercept=False) is used. What is the purpose of setting fit_intercept=False? If this were changed to True, how would you expect model2.coef_ and model2.intercept_ to be affected?

```
model2=LinearRegression(fit_intercept=False)
x=np.vstack([sample1,sample2,sample3])
model2.fit(x, y)
model2.coef_, model2.intercept_
```

```
(array([5.69493795e+00, 3.36972233e+00, 4.20919214e-03]), 0.0)
```

Question 4: Impact of Data Generation Parameters
In the first linear regression example (y=x*2 + 1 + 1*np.random.randn(n)), if the value 1 in 1*np.random.randn(n) were increased (e.g., changed to 5), how would you expect the fitted values of model.coef_ and model.intercept_ to change? Explain your reasoning.

```
np.random.seed(0)
n=20   # Number of data points
x=np.linspace(0, 10, n)
y=x*2 + 1 + 1*np.random.randn(n) # Standard deviation 1
print(x)
print(y)
```

```
[ 0.          0.52631579  1.05263158  1.57894737  2.10526316  2.63157895
  3.15789474  3.68421053  4.21052632  4.73684211  5.26315789  5.78947368
  6.31578947  6.84210526  7.36842105  7.89473684  8.42105263  8.94736842
  9.47368421 10.        ]
[ 2.76405235  2.45278879  4.08400114  6.39878794  7.07808431  5.28588001
  8.26587789  8.21706384  9.31783378 10.88428271 11.67035936 14.03322088
 14.39261667 14.80588554 16.18070534 17.12314801 19.33618434 18.68957858
 20.26043612 20.14590426]
```